Claude Code 入門

aoki-taquan

2026 年 03 月 15 日

目次

1 はじめに 4
1.1 Claude Code とは 4
1.2 このガイドについて 4
1.3 対象読者 4
1.4 前提知識 4
2 インストール 5
2.1 動作要件 5
2.2 npm でインストール 5
2.3 認証方法 5
2.4 アップデート 5
2.5 Claude Code on the Web 6
3 基本操作 7
3.1 起動と終了 7
3.2 プロンプトの送信 7
3.3 ワンショットモード 7
3.4 会話の再開 7
3.5 コンテキストウィンドウ 8
3.6 モデルの選択 8
4 ファイル操作とコード編集 9
4.1 ファイルの読み込み 9
4.2 コード検索 9
4.3 ファイルの編集 9
4.4 新規ファイルの作成 9
4.5 複数ファイルにまたがる変更 9
4.6 権限の管理 10
5 コマンドと設定 11
5.1 スラッシュコマンド一覧 11
5.2 CLAUDE.md による設定 11
5.3 CLAUDE.md の配置場所 12
5.4 設定ファイル 12
5.5 環境変数 12
6 Git 連携 13
6.1 基本的な Git 操作 13
6.2 コミットメッセージの自動生成 13
6.3 プルリクエストの作成 13
6.4 コードレビュー 13
6.5 ブランチ操作 13
6.6 Git の安全性 13
7 MCP（Model Context Protocol） 15

7.1 MCP とは 15
7.2 MCP サーバーの設定 15
7.3 MCP サーバーの追加 15
7.4 MCP サーバーの管理 15
7.5 活用例 16
8 IDE 連携 17
8.1 VS Code 拡張機能 17
8.1.1 インストール 17
8.1.2 主な機能 17
8.2 JetBrains プラグイン 17
8.2.1 インストール 17
8.2.2 使い方 17
8.3 ターミナルからの利用 17
9 高度な使い方 18
9.1 Hooks（フック） 18
9.1.1 フックの種類 18
9.1.2 設定例 18
9.2 ヘッドレスモード 18
9.2.1 出力フォーマット 19
9.3 マルチターンのパイプライン 19
9.4 カスタムスラッシュコマンド 19
9.5 Claude Agent SDK 19
10 Tips と ベストプラクティス 21
10.1 効果的なプロンプトの書き方 21
10.2 キーボードショートカット 21
10.3 コスト管理 21
10.4 CLAUDE.md を活用する 21
10.5 よくある問題と対処法 22
10.5.1 Claude が古い情報を参照する 22
10.5.2 権限エラーが頻発する 22
10.5.3 コンテキストが大きすぎる 22

1 はじめに

1.1 Claude Code とは

Claude Code は、Anthropic が提供するターミナルベースの AI アシスタントツールです。コマンドラインから直接 Claude を呼び出し、コードの読み書き、コマンドの実行、Git 操作、 ファイル検索など、ソフトウェア開発に必要な作業を自然言語で指示できます。

主な特徴は以下の通りです：

- **ターミナルネイティブ**：エディタやブラウザを離れずに AI と対話できる
- **コードベース理解**：プロジェクト全体を把握し、文脈に沿った回答・編集を行う
- **ツール実行**：シェルコマンドの実行、ファイルの読み書き、Git 操作を直接行う
- **安全性**：ユーザーの許可なく破壊的な操作を行わない権限管理システム

1.2 このガイドについて

本書は Claude Code を初めて使う方を対象に、インストールから基本操作、 高度な活用法までを網羅的に解説する入門書です。

1.3 対象読者

- ターミナル操作に慣れている開発者
- AI を開発ワークフローに取り入れたい方
- Claude Code をこれから使い始める方

1.4 前提知識

- 基本的なターミナル操作（`cd`、`ls` など）
- Git の基本概念（コミット、ブランチなど）
- いずれかのプログラミング言語の基礎知識

2 インストール

2.1 動作要件

Claude Code を利用するには以下が必要です：

- **OS**：macOS 10.15 以降、Ubuntu 20.04+ / Debian 10+ などの Linux
- **Node.js**：18 以上
- **Anthropic アカウント**：API キーまたは Claude の有料プラン（Max / Team / Enterprise）

2.2 npm でインストール

もっとも一般的なインストール方法です：

```
npm install -g @anthropic-ai/claude-code
```

インストール後、任意のディレクトリで `claude` コマンドを実行します：

```
cd your-project
claude
```

初回起動時に認証が求められます。ブラウザが開くので、Anthropic アカウントでログインしてください。

2.3 認証方法

Claude Code は複数の認証方法に対応しています：

- **Anthropic アカウント（デフォルト）**：`claude` 起動時にブラウザ認証
- **API キー**：環境変数 `ANTHROPIC_API_KEY` を設定
- **Amazon Bedrock**：`CLAUDE_CODE_USE_BEDROCK=1` を設定
- **Google Vertex AI**：`CLAUDE_CODE_USE_VERTEX=1` を設定

API キーを使う場合は以下のように設定します：

```
export ANTHROPIC_API_KEY="sk-ant-..."
claude
```

2.4 アップデート

最新版へのアップデートは以下のコマンドで行います：

```
npm update -g @anthropic-ai/claude-code
```

2.5 Claude Code on the Web

ブラウザ上で Claude Code を利用する方法もあります。claude.ai にアクセスし、 Claude Code を有効にすると、Web 上のターミナル環境で同様の操作が可能です。ローカル環境のセットアップが不要で、すぐに試すことができます。

3 基本操作

3.1 起動と終了

プロジェクトディレクトリで `claude` を実行すると対話モードが始まります：

```
cd your-project
claude
```

終了するには以下のいずれかを使います：

- `/exit` と入力
- `Ctrl+C` を 2 回押す
- `Ctrl+D`（EOF）を送信

3.2 プロンプトの送信

起動後、自然言語でプロンプトを入力します。複数行のプロンプトを書きたい場合は `Shift+Enter` で改行できます。 `Enter` のみで送信されます。

```
> このプロジェクトの構成を説明して
```

3.3 ワンショットモード

対話モードに入らず、単発でコマンドを実行することもできます：

```
claude -p "このリポジトリのREADMEを要約して"
```

`-p`（`--print`）フラグを使うと、結果を標準出力に表示して終了します。パイプと組み合わせることも可能です：

```
cat error.log | claude -p "このエラーの原因を分析して"
```

3.4 会話の再開

前回の会話を再開するには `--continue` フラグを使います：

```
claude --continue
```

特定の過去の会話を選んで再開する場合は `--resume` を使います：

```
claude --resume
```

3.5 コンテキストウィンドウ

Claude Code は会話が長くなるとコンテキストウィンドウの上限に近づきます。 その場合、古いメッセージは自動的に要約・圧縮されます。 `/compact` コマンドで手動で圧縮することもできます。

3.6 モデルの選択

デフォルトでは最新の Claude モデルが使用されます。 `/model` コマンドでモデルを切り替えることができます。

また、 `--model` フラグで起動時に指定することも可能です：

```
claude --model claude-sonnet-4-5-20250514
```

4 ファイル操作とコード編集

4.1 ファイルの読み込み

Claude Code はプロジェクト内のファイルを直接読み込むことができます。

```
> src/main.ts の内容を見せて
```

Claude は `Read` ツールを使ってファイルを読み取ります。画像ファイル（PNG、JPG など）や PDF も読み込み可能です。

4.2 コード検索

プロジェクト内のコードを検索する際は、ファイル名パターンや内容で検索できます：

```
> "handleSubmit" という関数がどこで定義されているか探して
```

Claude は内部的に `Glob`（ファイル名検索）と `Grep`（内容検索）ツールを使い分けます。

4.3 ファイルの編集

コードの修正を依頼すると、Claude は `Edit` ツールで差分ベースの編集を行います：

```
> handleSubmit 関数にバリデーションを追加して
```

編集前にファイルの内容を確認し、最小限の変更で修正を行います。 編集内容は差分（diff）として表示され、ユーザーが確認・承認できます。

4.4 新規ファイルの作成

新しいファイルの作成も可能です：

```
> src/utils/validation.ts にメールアドレスのバリデーション関数を作って
```

Claude は `Write` ツールを使ってファイルを作成します。

4.5 複数ファイルにまたがる変更

リファクタリングのように複数ファイルに影響する変更も得意です：

```
> UserService クラスを UserRepository と UserUseCase に分割して
```

Claude はプロジェクト全体の構造を理解した上で、関連するファイルを一括で変更します。

4.6 権限の管理

ファイルの読み書きやコマンドの実行には権限が必要です。 Claude が操作を行う前にユーザーに確認を求めます：

- **Allow once**：今回のみ許可
- **Allow for this session**：このセッション中は許可
- **Deny**：拒否

`/permissions` コマンドで現在の権限設定を確認・変更できます。

5 コマンドと設定

5.1 スラッシュコマンド一覧

Claude Code では `/` で始まるスラッシュコマンドが利用できます：

コマンド	説明
`/help`	ヘルプを表示
`/exit`	Claude Code を終了
`/clear`	会話履歴をクリア
`/compact`	会話を要約して圧縮
`/model`	使用するモデルを切り替え
`/permissions`	権限設定を管理
`/status`	現在のステータスを表示
`/cost`	セッションのトークン使用量とコストを表示
`/doctor`	インストール状態を診断
`/config`	設定を変更
`/mcp`	MCP サーバーの状態を確認
`/review`	コードレビューを実行
`/init`	CLAUDE.md を初期化

5.2 CLAUDE.md による設定

`CLAUDE.md` はプロジェクトのルートに置く設定ファイルです。 Claude Code が作業する際の規約や手順を自然言語で記述します：

```
# CLAUDE.md

## プロジェクト概要
TypeScript + React のフロントエンドアプリケーション。

## ビルド
npm run build

## テスト
npm test

## 規約
- 関数名はキャメルケース
- コンポーネントは関数コンポーネントで書く
```

`/init` コマンドで Claude に自動生成させることもできます。

5.3 CLAUDE.md の配置場所

CLAUDE.md は複数の場所に配置でき、それぞれスコープが異なります：

- **プロジェクトルート**（`./CLAUDE.md`）：チーム共有の設定
- **`.claude/` ディレクトリ内**（`.claude/CLAUDE.md`）：個人設定（`.gitignore` 推奨）
- **ホームディレクトリ**（`~/.claude/CLAUDE.md`）：全プロジェクト共通の個人設定

5.4 設定ファイル

`/config` コマンドまたは設定ファイルで動作をカスタマイズできます。 主な設定項目：

- **権限モード**：`allowedTools` で自動許可するツールを指定
- **テーマ**：ターミナルのカラーテーマ
- **通知**：タスク完了時の通知設定

5.5 環境変数

Claude Code の動作に影響する主な環境変数：

変数	説明
`ANTHROPIC_API_KEY`	API キー
`CLAUDE_CODE_USE_BEDROCK`	Amazon Bedrock を使用
`CLAUDE_CODE_USE_VERTEX`	Google Vertex AI を使用
`ANTHROPIC_MODEL`	デフォルトモデルを指定
`CLAUDE_CODE_MAX_TURNS`	最大ターン数を制限

6 Git 連携

6.1 基本的な Git 操作

Claude Code は Git リポジトリ内で動作している場合、Git 操作を直接実行できます：

```
> 変更をコミットして
```

Claude は `git status` で変更を確認し、適切なコミットメッセージを生成してコミットします。

6.2 コミットメッセージの自動生成

変更内容を分析して意味のあるコミットメッセージを自動生成します：

```
> 今の変更をコミットして。日本語でメッセージを書いて
```

Claude は差分を確認し、変更の意図を反映したメッセージを作成します。

6.3 プルリクエストの作成

GitHub の PR 作成も支援します：

```
> この変更でPRを作成して
```

Claude は `gh` コマンドを使い、変更内容をまとめたタイトルと説明を生成して PR を作成します。

6.4 コードレビュー

既存の PR のレビューも可能です：

```
> PR #42 をレビューして
```

`/review` コマンドで現在のブランチの変更をレビューすることもできます。

6.5 ブランチ操作

ブランチの作成や切り替えも自然言語で指示できます：

```
> feature/add-auth というブランチを作って切り替えて
```

6.6 Git の安全性

Claude Code は Git 操作に関して以下の安全方針を持っています：

- `git push --force` や `git reset --hard` などの破壊的操作は確認を求める
- コミット前に変更内容を表示して確認を促す
- `.env` などの機密ファイルをコミットしないよう注意する
- 基本的に新しいコミットを作成し、`--amend` は明示的に指示された場合のみ使用する

7 MCP（Model Context Protocol）

7.1 MCP とは

MCP（Model Context Protocol）は、AI モデルが外部のツールやデータソースと 連携するための標準プロトコルです。Claude Code は MCP クライアントとして動作し、 MCP サーバーが提供するツールを利用できます。

7.2 MCP サーバーの設定

MCP サーバーは設定ファイルで定義します。プロジェクトローカルの設定は `.claude/mcp.json` に記述します：

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@anthropic-ai/mcp-filesystem",
        "/path/to/directory"
      ]
    }
  }
}
```

グローバル設定は `~/.claude/mcp.json` に配置します。

7.3 MCP サーバーの追加

`claude mcp add` コマンドで対話的にサーバーを追加できます：

```
# ローカル（プロジェクト）スコープで追加
claude mcp add my-server -s local -- npx my-mcp-server

# グローバルスコープで追加
claude mcp add my-server -s global -- npx my-mcp-server
```

7.4 MCP サーバーの管理

```
# サーバー一覧を表示
claude mcp list
```

```
# サーバーの詳細を表示
claude mcp get my-server

# サーバーを削除
claude mcp remove my-server
```

対話モード内では `/mcp` コマンドでサーバーの状態を確認できます。

7.5 活用例

MCP サーバーを活用すると、以下のような連携が可能になります：

- **データベース**：SQL クエリの実行、スキーマの確認
- **Web 検索**：最新の技術ドキュメントの参照
- **外部 API**：Slack、GitHub Issues などとの連携
- **ファイルシステム**：特定ディレクトリへのアクセス制御

8 IDE 連携

8.1 VS Code 拡張機能

Claude Code は VS Code の拡張機能として利用できます。VS Code のターミナル内で `claude` を実行するか、 拡張機能をインストールして統合環境を利用します。

8.1.1 インストール

VS Code の拡張機能マーケットプレイスで “Claude Code” を検索してインストールします。

8.1.2 主な機能

- VS Code のターミナルパネル内での対話
- エディタで開いているファイルのコンテキスト共有
- ステータスバーでの状態表示
- コマンドパレットからの操作

8.2 JetBrains プラグイン

IntelliJ IDEA、WebStorm、PyCharm などの JetBrains IDE でも利用可能です。

8.2.1 インストール

JetBrains IDE のプラグインマーケットプレイスから “Claude Code” をインストールします。

8.2.2 使い方

IDE のターミナルパネルで `claude` を実行するか、プラグインが提供するパネルから直接操作します。

8.3 ターミナルからの利用

IDE を使わない場合でも、任意のターミナルエミュレータから利用できます。 tmux や screen と組み合わせると、バックグラウンドで長時間のタスクを 実行させることも可能です：

```
# tmux セッション内で実行
tmux new-session -s claude
claude
```

9 高度な使い方

9.1 Hooks（フック）

Hooks は Claude Code のツール実行の前後にカスタムスクリプトを実行する仕組みです。コード品質のチェックや自動フォーマットなどに活用できます。

9.1.1 フックの種類

- **PreToolUse**：ツール実行前に実行される
- **PostToolUse**：ツール実行後に実行される
- **Notification**：通知イベント時に実行される
- **SessionStart**：セッション開始時に実行される

9.1.2 設定例

`.claude/settings.json` にフックを定義します：

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command": "npx prettier --write $CLAUDE_FILE_PATH"
          }
        ]
      }
    ]
  }
}
```

この例では、ファイルの書き込みや編集の後に自動的に Prettier でフォーマットします。

9.2 ヘッドレスモード

CI/CD パイプラインやスクリプトから Claude Code を非対話的に実行できます：

```
claude -p "テストを実行して結果を報告して" \
  --output-format json \
  --max-turns 10
```

9.2.1 出力フォーマット

- `text`：プレーンテキスト（デフォルト）
- `json`：JSON 形式で出力
- `stream-json`：ストリーミング JSON

9.3 マルチターンのパイプライン

複数のプロンプトを連鎖させて複雑なワークフローを構築できます：

```
# 最初のプロンプト
claude -p "このプロジェクトの問題点を分析して" \
  --output-format json > analysis.json

# 結果を使って次のプロンプト
cat analysis.json | claude -p "この分析をもとに修正して" \
  --continue
```

9.4 カスタムスラッシュコマンド

プロジェクト固有のスラッシュコマンドを定義できます。`.claude/commands/` ディレクトリにマークダウンファイルを配置します：

```
.claude/commands/review.md
```

ファイルの内容がコマンドのプロンプトテンプレートになります：

```
以下の観点でコードレビューを行ってください：
1. セキュリティ上の問題
2. パフォーマンスの問題
3. 可読性の改善点

対象: $ARGUMENTS
```

`/project:review src/auth.ts` のように呼び出せます。

9.5 Claude Agent SDK

Claude Code の機能をプログラムから利用するための SDK です。カスタムエージェントの構築に使用できます：

```
import { Agent } from "claude-agent-sdk";

const agent = new Agent({
  model: "claude-sonnet-4-5-20250514",
});
```

```
const result = await agent.run("テストを実行して");
```

10 Tips と ベストプラクティス

10.1 効果的なプロンプトの書き方

Claude Code を効果的に使うためのプロンプトのコツ：

- **具体的に指示する**：「コードを改善して」より「handleSubmit 関数に null チェックを追加して」
- **コンテキストを与える**：「このプロジェクトは React + TypeScript で書かれています」
- **段階的に進める**：大きなタスクは複数のステップに分割して指示する
- **ファイルパスを明示する**：「src/utils/auth.ts の validateToken 関数を修正して」

10.2 キーボードショートカット

対話モードで使える主なショートカット：

ショートカット	動作
`Enter`	メッセージを送信
`Shift+Enter`	改行（送信せずに次の行へ）
`Ctrl+C`	現在の生成を中断 / 2 回で終了
`Ctrl+D`	セッションを終了
`Escape`	現在の入力をキャンセル
`上矢印`	直前のプロンプトを呼び出す

10.3 コスト管理

Claude Code はトークンを消費します。コストを把握するためのヒント：

- `/cost` コマンドでセッション中の使用量を確認
- `--max-turns` で自動実行のターン数を制限
- `/compact` で会話を圧縮してトークンを節約
- 大きなファイルは全体ではなく必要な部分だけ読ませる

10.4 CLAUDE.md を活用する

プロジェクト固有の情報を CLAUDE.md に記述しておくと、 毎回同じ説明をする必要がなくなります：

- ビルドコマンド、テストコマンド
- コーディング規約
- ディレクトリ構成の説明
- 頻繁に使うワークフロー

10.5 よくある問題と対処法

10.5.1 Claude が古い情報を参照する

Claude のトレーニングデータには期限があります。最新の API ドキュメントなどは、直接ファイルを渡すか URL を指定してください：

```
> https://docs.example.com/api の内容を参考にして実装して
```

10.5.2 権限エラーが頻発する

`/permissions` でよく使うツールを許可リストに追加すると、確認ダイアログの頻度を減らせます。

10.5.3 コンテキストが大きすぎる

`/compact` で会話を圧縮するか、`/clear` で新しい会話を開始してください。1 つの会話で多くのタスクを詰め込みすぎないことが重要です。